PROGRAMMING METHODS - LABORATORY 8

Program 01

Write a program that solves the general knapsack problem using dynamic programming.

Input

A text file with any name (standard input) has the following format:

- The first line contains the integer Mmax, specifying the carrying capacity of the backpack.
- Each of the next n lines contains three values P_i(m_i, c_i), i = 1, 2, ..., n, where P_i is an item, m_i is the mass of the i-th item, and c_i is the price of the i-th item.

Output

Output the table of values P_{i,j} for the best backpack packings with capacity j using item types from 1 to i, for i = 1, 2, ..., n and j = 1, 2, ..., Mmax. Also output the associated table Q_{i,j}, which indicates items P_i packed into the backpack in the last move.

Notes

- Divide the program into thematically grouped files.
- Remember to add comments to the code.
- Protect the program so that it handles exceptions.

Example

Pack six items into a backpack of capacity ten units; each item type is available in any quantity.

i	P_i	c_i [PLN]	m_i [unit]
1	Shirt	75	7
2	Trousers	150	8
3	Sweatshirt	250	6
4	Cap	35	4
5	Scarf	10	3
6	Sports shoes	100	9

Program 02

Write a program that solves the knapsack problem using recursion. The program should find a sequence of elements whose total weight is exactly equal to the backpack capacity (the selected elements exactly fill the backpack). The program should allow operations on a backpack with a given capacity and a given number of elements with different weights placed in the backpack.

Input

A text file with any name (standard input) has the following format:

- The first line contains a positive integer z (1 <= z <= 10^6), denoting the number of data sets.
- Each data set contains: an integer n (1 <= n <= 10^6) denoting the backpack capacity; an integer k (1 <= k <= 10^6) denoting the number of elements that may fill the backpack; and k non-repeating integers a1, ..., ak (1 <= ai <= 10^6), for i = 1 to k, representing the weights of consecutive elements.

Output

If there is no required solution, the program prints the word BRAK. Otherwise, for each data set with an existing solution, the program prints one line containing, in order:

- the backpack capacity;
- the equals sign "=" surrounded by single spaces;

- a sequence of all elements separated by single spaces, preserving input order and summing to the required capacity.

Notes

- Divide the program into thematically grouped files.
- Remember to add comments to the code.
- The recursive function must not contain loops.
- Protect the program so that it handles exceptions.

Example

Input

```
3
20
5
11 8 7 6 5
21
3
5 6 7
20
5
11 7 6 8 5
```

Output

```
20 = 8 7 5
BRAK
20 = 7 8 5
```